

Assignment01_Lentz.pdf

Piper Lentz

September 2026

Problem 1

Understand the problem

Goal: Sort intervals by start time and merge any overlapping intervals

Inputs: Array of intervals formatted [start, end]

Outputs: Array of sorted, non-overlapping intervals.

Requirements: intervals sorted by start value in ascending order, if next interval starts before current interval ends, merge. Use the smaller start and larger end for merged interval. Keep non-overlapping intervals separate. Don't lose or duplicate any interval coverage.

Design the solution

To preface my designs+code for all problems in this assignment, I want to let you know this is my first ever true computer science class (I majored in physics and astronomy in undergrad where I learned Python applications). I never learned any sorting algorithms until we went over bubble sort in class. Therefore, I will be using bubble sort as my sorting mechanism for this assignment - it is the only one I know and feel confident I can explain. I am excited to learn more techniques in this class!

1. Sort intervals in ascending order by start time. This will make it so we only have to compare adjacent intervals to determine if they need to be merged.
2. Go through intervals, and if current interval has a end time greater than or equal to the start time of the next interval, merge the intervals. If they don't overlap, make the next interval the new current interval.

Correctness

This algorithm works by sorting intervals by start time. It sets a "current interval" equal to the first interval in intervals. If the next interval in intervals has a start time less than or equal to the end time of current interval, it changes the current interval end time to the max between the current interval end time and the next interval end time. On the other hand, if the next interval in

intervals has a start time greater than the current interval's end time (aka there is a gap), end the current interval and make a new current interval.

If input is empty, the algorithm will return an empty list. If there is a duplicate interval, the interval and its duplicate will be merged together.

Pseudocode

```
function mergeIntervals(intervals):
    # sort by start time
    n = length of intervals
    if intervals is empty:
        return empty list
    end if statement
    for i from 0 to n-1:
        for j from 0 to n-2-i:
            if intervals[j].start > intervals[j+1].start:
                swap(intervals[j], intervals[j+1])
    # merge overlapping intervals
    mergedIntervals = []
    currentInterval = intervals[0]
    for i from 1 to n-1:
        if currentInterval.end >= intervals[i].start:
            currentInterval.end = max(currentInterval.end, intervals[i].end)
        else:
            append currentInterval to mergedIntervals
            currentInterval = intervals[i]
    append currentInterval to mergedIntervals
    return mergedIntervals
```

Analyze complexity

$O(n^2)$ because of the nested for loop

Tests and execution

```
Choose a problem: 1
-- Merge Intervals --
Enter intervals separated by semicolons, as: start end
Example: 1 3; 2 6; 8 10; 15 18
Input: 1 3; 2 6; 8 10; 15 18
Merged intervals: [1,6] [8,10] [15,18]
```

```
Choose a problem: 1

-- Merge Intervals --
Enter intervals separated by semicolons, as: start end
Example: 1 3; 2 6; 8 10; 15 18
Input: 0 0; 0 0
Merged intervals: [0,0]
```

Problem 2a

Understand the problem

Goal: All values in array grouped together in ascending order

Inputs: Array of integers 0,1,2

Outputs: Array of integers 0,1,2 sorted in place

Requirements: All 0s, 1s, 2s must be grouped together. Use a count-based solution. Array must be sorted in place. Do not duplicate or lose any values.

Design the solution

1. Count the number of 0s, 1s, 2s in the input.
2. For the first count of 0s indices in the array, enter 0 as the value.
3. Then, enter the count of 1s of 1 into the array.
4. Finally, add the count of 2s of 2 into the array. Now, we will have an array of 0s, 1s, 2s, in order and with the right frequencies.

Correctness

This algorithm checks each value in an array and determines if the value is a 0,1, or 2. If it isn't one of these values, it shows not a valid input. Then, it overwrites the array with the frequency of 0s, then 1s, then 2s in order (ex. The first count0 indices will be 0).

If the input is empty, return an empty list. Duplicate values will be treated as the value they are aka a 0, 1 or 2. An array of all 0s for example will return an array of all 0s.

Pseudocode

```
function group(values):
    count0 = 0
    count1 = 0
    count2 = 0
    n = length of values
    if values is empty
        return empty list
    end if statement
```

```

# counting 0s, 1s, 2s
for i from 0 to n-1:
    if values[i] == 0:
        count0 += 1
    elif values[i] == 1:
        count1 += 1
    elif values[i] == 2:
        count2 += 1
    else:
        print not a valid input
# overwrite in place
index = 0
for i from 1 to count0:
    values[index]=0
    index += 1
for i from 1 to count1:
    values[index] = 1
    index += 1
for i from 1 to count2:
    values[index] = 2
    index += 1

```

Analyze complexity

$O(n)$ because we only have one for loop that looks at each value.

Tests and execution

```

Choose a problem: 2a

-- Group 0s, 1s, and 2s - counting-based --
Enter values (only 0, 1, 2) separated by spaces.
Example: 2 0 1 2 0 1 0
Input: 0 1 2 2 2 2 0 0 1 1 1 1 0 2
Grouped result: 0 0 0 0 1 1 1 1 1 2 2 2 2 2

```

```

Choose a problem: 2a

-- Group 0s, 1s, and 2s - counting-based --
Enter values (only 0, 1, 2) separated by spaces.
Example: 2 0 1 2 0 1 0
Input: 1 1 1 1 1 1 1 1
Grouped result: 1 1 1 1 1 1 1 1

```

Problem 2b

Understand the problem

Goal: All values in array grouped together in ascending order

Inputs: Array of integers 0,1,2

Outputs: Array of integers 0,1,2 sorted in place

Requirements: All 0s, 1s, 2s must be grouped together. Use Dutch national flag solution. Array must be sorted in place. Do not duplicate or lose any values.

Design the solution

1. Create 3 indices, one designating a low (0) region boundary, one designating a mid region (the value we are observing now - put 1s behind this but after the low region), and one designating a high region (2) boundary.
2. Determine if the value at mid is a 0, 1 or 2.
3. If 0, pop it in the low region. If 1, move up to the next index (the 1 will stay behind it in a 1 region). If 2, pop it in the high region.

Correctness

This algorithm checks whether the value is a 0, 1, or 2, and places it in the region it belongs (0=low, 1=mid, 2=high). We can do this sorting in one pass because the values get pushed to where they belong immediately.

If input is empty, return an empty array. Duplicates are treated as their own value. An array of all 0s will return an array of all 0s.

Pseudocode

```
function Dflag(values):
    n = length of values
    if values is empty:
        return Values is empty
    end if statement
    low = 0
    mid = 0
    high = n-1
    while mid <= high:
        if values[mid] == 0:
            swap(values[mid], values[low])
            mid += 1
            low += 1
        elif values[mid] == 1:
            mid += 1
        elif values[mid] == 2:
            swap(values[mid], values[high])
```

```

        high -= 1
    else:
        print "Not a valid input"
    return values

```

Analyze complexity

$O(n)$ because we only evaluate each value once (one for loop).

Tests and execution

```

Choose a problem: 2b

-- Dutch National Flag --
Enter values (only 0, 1, 2) separated by spaces.
Example: 2 0 1 2 0 1 0
Input: 1 0 2 0 0 1 0 2 2 2 1
Result: 0 0 0 0 1 1 1 2 2 2 2

```

```

=====
Choose a problem: 2b

-- Dutch National Flag --
Enter values (only 0, 1, 2) separated by spaces.
Example: 2 0 1 2 0 1 0
Input: 1 1 1 1 1 1 1 1
Result: 1 1 1 1 1 1 1 1

```

Problem 3

Understand the problem

Goal: Arrange numbers so that their concatenation becomes the largest possible number

Inputs: Vector of non-negative integers

Outputs: String of the highest possible concatenation

Requirements: Sort numbers so that their concatenation is largest number possible. Numbers must be non-negative. Do not lose or duplicate input values. Return the largest possible concatenation as a string.

Design the solution

1. We need our result in a string, so first go through every value in the vector and make it a string.

2. Use bubble sort method to put the strings in the order. Decide if two strings need to swap places by comparing whether the $A[i]+A[i+1]$ value or $A[i+1]+A[i]$ value is greater.
3. Once in the correct order, concatenate the strings.

Correctness

This algorithm returns the largest possible concatenated number because it sorts the values using a decision to swap based on which concatenated value is greater ($A[i]+A[i+1]$ value or $A[i+1]+A[i]$ value). It repeats this sorting algorithm so that we can be sure every string is in the correct index.

If there is an empty input, an empty vector will be returned. Duplicates in the data will be treated as their own value.

Pseudocode

```
function largestNum(values):
    if values is empty:
        return empty vector
    end if statement
    # make each value into a string
    valuesStr = []
    n = length of values
    for i from 0 to n-1:
        append string of values[i] to valuesStr
    # sort
    for i from 0 to n-2:
        for j from 0 to n-2-i:
            if (valuesStr[j]+valuesStr[j+1]<valuesStr[j+1]+valuesStr[j]):
                swap(valuesStr[j], valuesStr[j+1])
    # concatenate
    result = []
    for i from 0 to n-1:
        result = result + valuesStr[i]
    return result
```

Analyze complexity

$O(n^2)$ because of the nested for loop in bubble sort.

Tests and execution

```
Choose a problem: 3

-- Largest Number --
Enter non-negative integers separated by spaces.
Example: 3 30 34 5 9
Input: 2 3 5 11 444 63 74
Largest number: 746354443211
```

```
Choose a problem: 3

-- Largest Number --
Enter non-negative integers separated by spaces.
Example: 3 30 34 5 9
Input: 1 1 1 1
Largest number: 1111
```

Problem 4

Understand the problem

Goal: Group anagrams together

Inputs: Vector of strings

Outputs: Vector of strings with anagrams grouped together

Requirements: Anagrams grouped together. Words in each anagram group retain original order in the vector. Do not lose or duplicate any words. Assumes no spaces or punctuation.

Design the solution

1. Create a map that will keep track of every key in the vector and its frequency.
2. Go through the words in the vector, and create a key based on the word with its characters in alphabetical order (ex. "eat" becomes "aet"). This way, anagrams will all have the same key.
3. Place words in their correct key in the map.
4. In another vector, go through the map and append words from each key. This will make anagrams be grouped together and the words appearing in their original order within the anagram group.

Correctness

This algorithm creates a map with keys based on the alphabetical lettering of words. If the input is empty, an empty vector will be returned. Words that appear in the original vector twice will be treated as separate words.

Pseudocode

```
function anagram(words):
    if words is empty:
        return empty vector
    end if statement
    n = length of words
    map = empty map
    for i from 0 to n-1:
        # sort every word's characters alphabetically
        key = sortCharacter(words[i])
        if key is not in map:
            map[key] = []
        append words[i] to map[key]
    # make a list with words with the same key grouped together
    result = []
    for every key in map:
        append map[key] to result
    return result
```

Analyze complexity

$O(\log n)$ since we insert values into a map. Early on, lots of keys may need to be made, and less later on. I also think there might be something with rearranging the letters in each word to make the key - I'm not sure how that would work.

Tests and execution

```
Choose a problem: 4
-- Group Anagrams --
Enter words separated by spaces.
Example: eat tea tan ate nat bat
Input: eat tea tan ate nat bat loop
Grouped anagrams: [bat] [eat tea ate] [tan nat] [loop]
```

```
Choose a problem: 4

-- Group Anagrams --
Enter words separated by spaces.
Example: eat tea tan ate nat bat
Input: peace meow dog roof
Grouped anagrams: [peace] [dog] [meow] [roof]
```

Problem 5

Understand the problem

Goal: Yes or no - do any meetings overlap?

Inputs: Vector of intervals organized [start, end]

Outputs: Boolean - True or False, answering if someone can attend every meeting

Requirements: Gives a yes or no answer. Meetings that begin exactly when another ends are allowed. Do not duplicate or lose any meeting intervals.

Design the solution

1. Sort the intervals by start time in ascending order. This will allow us to check for overlapping intervals more efficiently.
2. Next, check if any of the intervals overlap. Do this by seeing if the end time of the current interval is greater than the start time of the next interval.
3. Since the question only wants to know whether the person can attend all meetings, if there are any overlaps at all, we can return False, she cannot attend all meetings. If no overlaps, return True. We do not need to keep a running tab of which intervals overlap and which do not.

Correctness

This algorithm checks if any intervals in the vector overlap. It sorts them by starting time, then checks if any end time of interval i is later than the start time of interval $i+1$. If input is empty, the algorithm returns an empty vector. If there are two of the same intervals in the input, the algorithm will return false since the person cannot attend both of those meetings (they totally overlap).

Pseudocode

```
function meetingOverlap(intervals):
    if intervals is empty:
        return empty vector
    end if statement
    n = length of intervals
```

```

for i from 0 to n-1:
    for j from 0 to n-2-i:
        if intervals[j].start > intervals[j+1].start:
            swap(intervals[j], intervals[j+1])
# check if any intervals overlap
for i from 0 to n-2:
    if intervals[i].end > intervals[i+1].start:
        return False
return True

```

Analyze complexity

$O(n^2)$ because of bubble sort and nested for loops.

Tests and execution

```

Choose a problem: 5

-- Meeting Overlap --
Enter intervals separated by semicolons, as: start end
Example: 1 3; 5 6; 8 10
Input: 1 3; 5 9; 8 10
No overlaps: False

```

```

Choose a problem: 5

-- Meeting Overlap --
Enter intervals separated by semicolons, as: start end
Example: 1 3; 5 6; 8 10
Input: 1 2; 1 2
No overlaps: False

```

Problem 6a

Understand the problem

Goal: Find the closest k points to the origin

Inputs: Points in the form (x,y), and an integer

Outputs: List of k points in form (x,y)

Requirements: The k closest points to the origin are returned. Calculate distance to origin with Pythagorean theorem. Use a sorting based solution.

Design the solution

1. Make sure k is a positive integer

2. Calculate the distances from each point in the input using Pythagoreans Theorem. We can just do $x^2 + y^2$ (not taking the root of that) because it gives us an equivalent way to sort the distances from smallest to largest, and it reduces the amount of steps the algorithm needs to take.
3. Place these distances and associated points in a list.
4. Use bubble sort to sort the distances in ascending order. Then for indices 0 to $k-1$ in the list (because that will give us the k closest points), append the points to another list.

Correctness

This algorithm calculates the distance to the origin for each point, then sorts the points by distance in ascending order such that closest points are at the beginning of the list. It then takes the first k points from that list and returns them. If given empty input, it returns an empty list. If there are two or more of the same point in the input, this algorithm treats them as separate points. If k is larger than the number of points, algorithm returns all points and lets the user know.

Pseudocode

```
function closePoints(points, k):
    if points is empty:
        return empty list
    end if statement
    if k is 0 or negative or not an integer:
        return Make k a positive integer
    end if statement
    n = length of points
    key = []
    # calculate distances
    for i from 0 to n-1:
        dist = (points[i].x**2 + points[i].y**2)
        append dist, points[i] to key
    # sort based on dist
    for i from 0 to n-1:
        for j from 0 to n-2-i:
            if key[j].dist > key[j+1].dist:
                swap(key[j], key[j+1])
    # return k closest points
    result = []
    for i from 0 to k-1:
        append key[i].points
    return result
```

Analyze complexity

$O(n^2)$ bubble sort dominates.

Tests and execution

```
Choose a problem: 6a

-- K Closest Points --
Enter points separated by semicolons, as: x y
Example: 1 3; -2 2; 5 8; 0 1
Input: 4 2; 44 2; -33 0
Enter k: 2
Closest points: (4,2) (-33,0)
```

```
Choose a problem: 6a

-- K Closest Points --
Enter points separated by semicolons, as: x y
Example: 1 3; -2 2; 5 8; 0 1
Input: 0 0; 0 0; 0 0
Enter k: 2
Closest points: (0,0) (0,0)
```

Problem 6b

Understand the problem

Goal: Find the closest k points to the origin

Inputs: Points in the form (x,y), and an integer

Outputs: List of k points in form (x,y)

Requirements: The k closest points to the origin are returned. Calculate distance to origin with Pythagorean theorem. Use heap-based solution.

Design the solution

1. Make sure k is a positive integer
2. Create a max heap. It will keep track of the largest value in the heap.
3. Calculate the distances from each point in the input using Pythagoreans Theorem. We can just do x squared + y squared (not taking the root of that) because it gives us an equivalent way to sort the distances from smallest to largest, and it reduces the amount of steps the algorithm needs to take.

4. Go through the distances. If the size of the heap is not yet k , place the distance and associated point in the heap. Then when the heap size reaches k , we need to compare the distance of the current point to the max distance in the heap. If the current point's distance is greater, pop out the top distance in the heap, and put the current point/distance in.
5. After determining which points/distances belong in the heap of size k (aka the k closest points), put those points in a list. These are the k closest points.

Correctness

This algorithm calculates the distance to the origin for each point, then maintains a heap of the smallest k distances and associated points. If given empty input, it returns an empty list. If there are two or more of the same point in the input, this algorithm treats them as separate points. If k is larger than the number of points, algorithm returns all points and lets the user know.

Pseudocode

```
function closePoints(points, k):
    if points is empty:
        return Points is empty
    end if statement
    if k is 0 or negative or not an integer:
        return Make k a positive integer
    end if statement
    n = length of points
    heap = empty max-heap
    for i from 0 to n-1;
        dist = points[i].x**2 + points[i].y**2
        if size of heap < k:
            append(dist, points[i]) to the heap
        elif dist < largest dist in heap
            pop top of heap # pop out largest dist in heap
            append (dist, points[i]) to heap
    # append k points
    result = []
    while heap is not empty:
        append points from heap to result in descending order
    return result
```

Analyze complexity

$O(n \log k)$ because each point does one push or pop on the heap and each push/pop is $O(\log k)$.

Tests and execution

```
Choose a problem: 6b

-- K Closest Points to Origin --
Enter points separated by semicolons, as: x y
Example: 1 3; -2 2; 5 8; 0 1
Input: 1 3; -2 2; 5 8
Enter k: 2
Closest points: (-2,2) (1,3)
```

```
Choose a problem: 6b

-- K Closest Points to Origin --
Enter points separated by semicolons, as: x y
Example: 1 3; -2 2; 5 8; 0 1
Input: 0 0; 0 0; 0 0
Enter k: 2
Closest points: (0,0) (0,0)
```

Problem 7

Understand the problem

Goal: Return k most frequent values in array

Inputs: Integer array

Outputs: k most frequent integers

Requirements: Returns k most frequent points in array. If tie, return the smaller value first. Do not duplicate values or lose values.

Design the solution

1. Create a map that associates a number with its frequency in the input.
2. Go through every value in the array, creating a key that is the number itself. If the key hasn't been made yet, create the key and set the frequency to 1. If the key has been made, that number has been seen before in the array, so add 1 to the frequency.
3. Then, add every key (aka value) and its frequency to a list. Sort this list in descending order by frequency using bubble sort. If the frequency of two values are the same, put the smaller value first.
4. Place the first k values from this sorted list into another list. These are our k most frequent values.

Correctness

This algorithm uses a map to associate each value in the array with its frequency. It then sorts the values by frequency in descending order so that the values at the beginning are the most frequent. If the input is empty, the algorithm returns an empty array.

Pseudocode

```
function frequency(values, k):
    if values is empty:
        return Values is empty
    end if statement
    if k is < 0 or not an integer:
        return Make k a positive integer
    n = length of values
    # find frequency of integers in values
    map = empty map
    for i from 0 to n-1:
        key = values[i]
        if key is not in map:
            map[key] = 1
        else:
            map[key] +=1
    # copy map into list of (value, count) to sort
    pairs = []
    for every (key, count) in map:
        append (key, count) to pairs
    # sort in descending order by count
    m = length pairs
    for i from 0 to m-1:
        for j from 0 to m-1-i:
            if pairs[j].count < pairs[j+1].count or (pairs[j].count == pairs[j+1].count
and pairs[j].key > pairs[j+1].key)
                swap(pairs[j], pairs[j+1])
    # give first k values
    result = []
    for i from 0 to k-1:
        append pairs[i].key to result
    return result
```

Analyze complexity

$O(m^2)$ because of the nested for loop.

Tests and execution

```
Choose a problem: 7

-- Top K Frequent Elements --
Enter values separated by spaces.
Example: 1 1 1 2 2 3
Input: 2 2 2 4 5 5 5 5 7 8 8
Enter k: 3
Top 3 frequent elements: 5 2 8
```

```
Choose a problem: 7

-- Top K Frequent Elements --
Enter values separated by spaces.
Example: 1 1 1 2 2 3
Input: 1 2 3 4 5
Enter k: 1
Top 1 frequent elements: 1
```

Problem 8

Understand the problem

Goal: Return sorted array from k sorted array

Inputs: Sorted integer arrays, integer k

Outputs: One sorted array

Requirements: Returns one fully sorted array (combining the input arrays).

Solution must not repeatedly sort the complete output.

Design the solution

1. Create a min-heap that will keep track of the minimum value in the heap.
2. Since the input arrays are already sorted, we can place the first indices of each array into the heap.
3. While the heap is not empty, take the minimum in the heap, pop it out of the heap, and place it in a new array. This means that the minimum of the lowest values in each array get added to the new array first. Then,

from the array that had its value popped out of the heap, take the next element and place it in the heap.

Correctness

This algorithm uses a min-heap to sort the already sorted arrays into one giant sorted array. The min-heap traces through the elements in each array in order (which we know to be lowest to highest), and pops the smallest of the small values into another array, ensuring the new array is sorted. If an array in the input is empty, the algorithm tells the user there is an empty array.

Pseudocode

```
function sortSorted(arrays, k):
    result = []
    heap = empty min-heap
    # put first element of each array in heap
    for i from 0 to k-1:
        if arrays[i] is not empty:
            push(arrays[i][0], i, 0) into heap
        else:
            print array i is empty
    # get min of heap and put it in result
    while heap is not empty:
        (value, arrayidx, elementidx) = heap minimum
        append value to result
        pop that minimum element from heap
        # push next min (just the next value) from array into heap
        nextidx = elementidx + 1
        if nextidx < length of arrays[arrayidx]:
            push (arrays[arrayidx][nextidx], arrayidx, nextidx) into heap
    return result
```

Analyze complexity

$O(N \log k)$ because heap operation is $\log K$ and every element from every array is considered once by the heap.

Tests and execution

```
Choose a problem: 8
-- Merge K Sorted Arrays --
Enter each sorted array separated by semicolons.
Example: 1 4 7; 2 5 8; 3 6 9
Input: 2 3 4; 3 4 5; 5 6 7
Merged array: 2 3 3 4 4 5 5 6 7
```

```

Choose a problem: 8

-- Merge K Sorted Arrays --
Enter each sorted array separated by semicolons.
Example: 1 4 7; 2 5 8; 3 6 9
Input: 1 2; 3 3 3 3; 4 5 6
Merged array: 1 2 3 3 3 3 4 5 6

```

Problem 9

Understand the problem

Goal: Return the kth largest element from unsorted array

Inputs: Integer array, integer k

Outputs: Integer (the kth largest element from the array)

Requirements: Do not sort the whole array. Return the kth largest element. Do not lose or duplicate values in array.

Design the solution

1. Create a min-heap to keep track of the smallest value in the heap.
2. Go through the values in the array. If the size of the heap is not yet k, push the current value into the heap. When the heap is size k, compare the current value to the minimum in the heap. If the current value is larger than the minimum of the heap, pop out the heap's minimum and add the current value to the heap. This way, after going through every value in the array once, the heap will contain the k largest values from the array.
3. Return the minimum of the heap, which is the kth largest value in the array.

Correctness

This algorithm uses a min-heap to keep track of the largest values in the array. It pops out the minimum of the heap if a larger value is encountered. If the input is empty, the algorithm tells the user so.

Pseudocode

```

function kLargest(values, k):
    if values is empty:
        return Values is empty
    end if statement
    if k <= 0 or not an integer or larger than n:
        return Make k a positive integer and <= n
    heap = empty min-heap

```

```

# keep a heap size of k with largest values seen
for i in values:
    if size(heap) < k:
        push i into heap
    elif i > heap.min:
        heap.popMin()
        heap.push(i)
return heap.min

```

Analyze complexity

$O(n \log k)$ because each element does at most one push or pop on the heap and the heap never grows past size k .

Tests and execution

```

Choose a problem: 9

-- Kth Largest Element --
Enter values separated by spaces.
Example: 3 2 1 5 6 4
Input: 3 44 5 7 65 3
Enter k: 3
The 3-th largest element is: 7

```

```

Choose a problem: 9

-- Kth Largest Element --
Enter values separated by spaces.
Example: 3 2 1 5 6 4
Input: 1 1 1 1 1
Enter k: 4
The 4-th largest element is: 1

```

Problem 10

Understand the problem

Goal: Count the number of inversions of an array.

Inputs: Array

Outputs: Integer of the number inversions necessary

Requirements: $O(n \log n)$ time. Returns number of inversions are needed to sort the array.

Design the solution

1. We will use a version of merge sort to count the inversions. So, this algorithm breaks down the list of values into blocks of 1 value (round 1), 2 values (round 2), 4 values(round 3) and so on until the full array becomes one block.
2. Within each round, we need pointers to indicate the values we are comparing. An i pointer indicates the value to compare in the left block, and a j pointer indicates a value to compare in the right block. If $\text{values}[i] \leq \text{values}[j]$, no inversion. If $\text{values}[j] < \text{values}[i]$, we add $\text{mid} - i + 1$ to the count of inversions because the left block is already sorted, so $\text{values}[j]$ is also smaller than every value from $\text{values}[i]$ to the end of the left block (indicated as mid).
3. After a merge, put the sorted result back into the array and double the block size.

Correctness

This algorithm uses a version of merge sort to count the inversions (explained above). If the input is empty, the algorithm tells the user values is empty.

Pseudocode

```
function countInversions(values):
    n = length of values
    temp = new array of size n
    count = 0
    width = 1
    while width < n:
        for low from 0 to n - width - 1, increasing low by 2 * width each time:
            mid = low + width - 1
            hi = min(low + 2 * width - 1, n - 1)
            i = low
            j = mid + 1
            k = low
            while i <= mid and j <= hi:
                if values[i] <= values[j]:
                    temp[k] = values[i]
                    k = k + 1
                    i = i + 1
                else:
```

```

        temp[k] = values[j]
        k = k + 1
        j = j + 1
        count = count + (mid - i + 1)
    while i <= mid:
        temp[k] = values[i]
        k = k + 1
        i = i + 1
    while j <= hi:
        temp[k] = values[j]
        k = k + 1
        j = j + 1
    copy temp[low..hi] back into values[low..hi]
    width = width * 2
    return count

```

Analyze complexity

$O(n \log n)$ because bottom up merge does about $\log n$ passes and each pass merges the whole array in $O(n)$.

Tests and execution

```

Choose a problem: 10

-- Inversion Count --
Enter values separated by spaces.
Example: 2 4 1 3 5
Input: 2 24 5 5 67 8 99 9
Number of inversions: 7

```

```

Choose a problem: 10

-- Inversion Count --
Enter values separated by spaces.
Example: 2 4 1 3 5
Input: 1 1 1 1 1
Number of inversions: 0

```